BLOCKCHAIN IDENTITY MANAGEMENT SYSTEM

PROJECT DOCUMENTATION

Student Details

Name:
Gorle Raju

Intern ID:
CIST737

Project Name

Blockchain-Based Identity Management System Using Python

Project Scope

The main scope of this project is to develop a decentralized identity management system using blockchain technology implemented in Python. This project stores user identity records (Name, Aadhaar, Email) as immutable blocks in a custom-built blockchain, demonstrating how distributed ledgers can secure sensitive personal data. The project is useful for learning blockchain fundamentals, SHA-256 hashing, chain integrity, and cybersecurity principles.

Introduction

A Blockchain-Based Identity Management System is a cybersecurity application that uses an immutable, hash-linked chain of blocks to store and protect identity records. Each block contains a user's identity data, a timestamp, the previous block's hash, and its own SHA-256 hash — making any tampering immediately detectable. This project implements the blockchain entirely in Python using the **hashlib** and **json** standard library modules, without any external blockchain framework.

Objectives

- Understand blockchain data structure and hash linking
- Implement SHA-256 hashing using Python's hashlib module
- Build a Block class with index, timestamp, data, and hash fields
- Build a Blockchain class with genesis block creation and chain management
- Accept and store user identity data (Name, Aadhaar, Email) on-chain
- Verify chain integrity through linked hashes
- Display the full blockchain with all block details

Technologies Used

Technology	Purpose
Python	Core Programming Language
hashlib (SHA-256)	Block Hash Generation
json module	Data Serialization for Hashing
time module	Timestamp Generation
Online Python Compiler	Program Execution / Testing

Python Code

```python
import hashlib
import json
import time

# Block Class
class Block:
    def __init__(self, index, timestamp, data, previous_hash):
        self.index = index
        self.timestamp = timestamp
        self.data = data
        self.previous_hash = previous_hash
        self.hash = self.calculate_hash()

    # Generate hash
    def calculate_hash(self):
        block_string = json.dumps({
            "index": self.index,
            "timestamp": self.timestamp,
            "data": self.data,
            "previous_hash": self.previous_hash
        }, sort_keys=True).encode()
        return hashlib.sha256(block_string).hexdigest()


# Blockchain Class
class Blockchain:
    def __init__(self):
        self.chain = [self.create_genesis_block()]

    def create_genesis_block(self):
        return Block(0, time.time(), "Genesis Block", "0")

    def get_latest_block(self):
        return self.chain[-1]

    def add_block(self, data):
        previous_block = self.get_latest_block()
        new_block = Block(
            len(self.chain),
            time.time(),
            data,
            previous_block.hash
        )
        self.chain.append(new_block)

    def display_chain(self):
        for block in self.chain:
            print("\n------------------------")
            print("Block Index  :", block.index)
            print("Timestamp    :", block.timestamp)
            print("Data         :", block.data)
            print("Previous Hash:", block.previous_hash)
            print("Hash         :", block.hash)
            print("------------------------")
```

```
# Main Program
identity_chain = Blockchain()

while True:
    print("\n===== Identity Management System =====")
    print("1. Add Identity")
    print("2. View Blockchain")
    print("3. Exit")

    choice = input("Enter choice: ")

    if choice == "1":
        name    = input("Enter Name: ")
        aadhaar = input("Enter Aadhaar Number: ")
        email   = input("Enter Email: ")
        identity_data = {
            "Name": name,
            "Aadhaar": aadhaar,
            "Email": email
        }
        identity_chain.add_block(identity_data)
        print("\nIdentity added successfully!")

    elif choice == "2":
        identity_chain.display_chain()

    elif choice == "3":
        print("Exiting...")
        break

    else:
        print("Invalid choice!")
```

Sample Input

```
===== Identity Management System =====
1. Add Identity
2. View Blockchain
3. Exit
Enter choice: 1
Enter Name: Gunturu Lakshmi Kumari
Enter Aadhaar Number: 1234-5678-9012
Enter Email: lakshmi@example.com
Identity added successfully!
```

Sample Output

```
===== Identity Management System =====
1. Add Identity
2. View Blockchain
3. Exit
Enter choice: 2

------------------------
Block Index  : 0
Timestamp    : 1716900000.123456
```

```
Data         : Genesis Block
Previous Hash: 0
Hash         : 0000a1b2c3d4e5f6...
------------------------

------------------------
Block Index  : 1
Timestamp    : 1716900060.654321
Data         : {'Name': 'Gunturu Lakshmi Kumari',
                'Aadhaar': '1234-5678-9012',
                'Email': 'lakshmi@example.com'}
Previous Hash: 0000a1b2c3d4e5f6...
Hash         : 00008f9e7d6c5b4a...
------------------------
```

Features of the Project

- Custom blockchain implementation entirely in Python
- SHA-256 cryptographic hashing for each block
- Genesis block automatically created on startup
- Accepts identity data: Name, Aadhaar Number, Email
- Immutable hash-linked chain — any tampering breaks the chain
- Interactive console menu (Add / View / Exit)
- Displays full block details including previous and current hash
- Beginner-friendly, no external blockchain libraries required

Advantages

- Easy to understand and extend
- Demonstrates real cryptographic concepts (SHA-256)
- Shows how blockchain prevents data tampering
- Useful for learning decentralized identity concepts
- No third-party dependencies — pure Python standard library

Limitations

- Data is stored only in memory (not persisted to disk)
- No Proof of Work or consensus mechanism implemented
- Single-node system — not a true distributed network
- No encryption of Aadhaar or personal data at rest
- No input validation for Aadhaar format or email format

Future Enhancements

- Add Proof of Work (PoW) mining for block validation
- Persist blockchain data to a JSON or SQLite file
- Add chain integrity verification method (is_chain_valid)
- Implement a peer-to-peer network for decentralization
- Encrypt sensitive fields (Aadhaar) before storing on-chain
- Build a web-based UI using Flask or Django
- Add digital signatures using public-key cryptography
- Integrate QR code generation for identity verification

Conclusion

The Blockchain-Based Identity Management System demonstrates how Python's standard library can be used to build a functional, hash-secured blockchain from scratch. By implementing the Block and Blockchain classes with SHA-256 hashing and hash linking, this project gives students hands-on experience with the core data structure that underpins technologies like Ethereum and Hyperledger. The project reinforces concepts of immutability, cryptographic integrity, and decentralized record-keeping that are central to modern cybersecurity and Web3 development.